ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 6

N15. Սենսորների չափումների վերլուծություն

Աշխատանքի նպատակ

Սովորել C++ լեզվում ստեղծել դասեր՝ կիրառելով private և public բաժինները, պահպանել օբյեկտների տվյալները, օգտագործել vector կոնտեյները և մշակել սենսորների չափումների հավաքածուն: Կիրառել դասի մեթոդներ՝ տվյալները ստուգելու, միջին արժեքը, նվազագույնն ու առավելագույնը հաշվելու համար:

Խնդրի պահանջ

- Ստեղծել Sensor դաս՝ հետևյալ դաշտերով՝ id, location, minAllowed, maxAllowed և readingCount:
- Ստուգել, որ minAllowed < maxAllowed պայմանը ճիշտ է:
- addReading() մեթոդով պահպանել չափման արժեքը միայն այն դեպքում, երբ այն թույլատրելի միջակայքում է:
- Հաշվել սենսորի չափումների միջին, նվազագույն և առավելագույն արժեքները:
- Գտնել ամենաշատ չափումներ գրանցած սենսորը և որոշել նրա առավելագույն չափումը:
- Սենսորները դասակարգել ըստ դրանց չափումների քանակի՝ աճման կարգով:

Տեսական մաս

C++ լեզվում դասը (class) օգտագործվում է տվյալներն ու դրանց հետ աշխատող մեթոդները միավորելու համար: private դաշտերը հասանելի են միայն դասի ներսում, ինչն օգնում է պաշտպանել տվյալները պատահական փոփոխություններից, իսկ public մեթոդները թույլ են տալիս անվտանգ կերպով աշխատել օբյեկտի հետ:

Կոնստրուկտորը սկզբնական արժեքներ է տալիս օբյեկտի դաշտերին: vector-ը դինամիկ զանգված է, որի չափը կարող է փոփոխվել ծրագրի աշխատանքի ընթացքում: Չափումները պահվում են վեկտորում, իսկ ցիկլերի և ստանդարտ ալգորիթմների միջոցով կարելի է հաշվել դրանց միջինը, գտնել նվազագույն և առավելագույն արժեքները կամ համեմատել սենսորները: Միջինը հաշվվում է բոլոր չափումների գումարը դրանց քանակին բաժանելով:

Եթե չափման արժեքը դուրս է սենսորի թույլատրելի սահմաններից, այն չի ավելացվում տվյալների ցանկին: Եթե չափումներ չկան, միջին, նվազագույն կամ առավելագույն արժեքները հաշվարկել հնարավոր չէ, ուստի ծրագիրը պետք է այդ դեպքը հաշվի առնի:

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <numeric>
#include <stdexcept>
#include <iomanip>
using namespace std;

class Sensor {
private:
    int id;
    string location;
    double minAllowed, maxAllowed;
    vector<double> readings;

public:
    Sensor(int sensorId, string sensorLocation,
           double minValue, double maxValue)
        : id(sensorId), location(sensorLocation),
          minAllowed(minValue), maxAllowed(maxValue) {
```

```
        if (minAllowed >= maxAllowed)
            throw invalid_argument("Invalid sensor limits");
    }

    bool addReading(double value) {
        if (value < minAllowed || value > maxAllowed)
            return false;
        readings.push_back(value);
        return true;
    }

    int getReadingCount() const {
        return static_cast<int>(readings.size());
    }

    int getId() const { return id; }
    string getLocation() const { return location; }

    double average() const {
        if (readings.empty()) return 0.0;
        double sum = accumulate(readings.begin(),
                                readings.end(), 0.0);
        return sum / readings.size();
    }

    double minimum() const {
        if (readings.empty()) return 0.0;
        return *min_element(readings.begin(), readings.end());
    }

    double maximum() const {
        if (readings.empty()) return 0.0;
        return *max_element(readings.begin(), readings.end());
    }

    void printInfo() const {
        cout << "ID: " << id
             << ", location: " << location
             << ", reading count: " << readings.size();

        if (!readings.empty()) {
            cout << ", average: " << average()
                 << ", minimum: " << minimum()
                 << ", maximum: " << maximum();
        } else {
            cout << ", no readings";
        }
        cout << '\\n';
    }
};

int main() {
    try {
        vector<Sensor> sensors;
```

```
        sensors.emplace_back(1, "Room A", 0, 50);
        sensors.emplace_back(2, "Room B", -10, 60);
        sensors.emplace_back(3, "Warehouse", 0, 100);

        sensors[0].addReading(22.5);
        sensors[0].addReading(24.0);
        sensors[0].addReading(55.0); // outside allowed range
        sensors[1].addReading(18.0);
        sensors[1].addReading(20.5);
        sensors[1].addReading(19.0);
        sensors[2].addReading(35.0);
        sensors[2].addReading(42.0);

        cout << fixed << setprecision(2);
        cout << "Sensor data:\\n";
        for (const Sensor& sensor : sensors)
            sensor.printInfo();

        auto mostRead = max_element(
            sensors.begin(), sensors.end(),
            [](const Sensor& a, const Sensor& b) {
                return a.getReadingCount() < b.getReadingCount();
            });

        cout << "\\nSensor with most readings: ";
        if (mostRead != sensors.end()) {
            cout << mostRead->getId()
                 << " (" << mostRead->getLocation() << ")\\n";
            if (mostRead->getReadingCount() > 0)
                cout << "Its maximum reading: "
                     << mostRead->maximum() << '\\n';
        }

        sort(sensors.begin(), sensors.end(),
             [](const Sensor& a, const Sensor& b) {
                 return a.getReadingCount() < b.getReadingCount();
             });

        cout << "\\nSensors sorted by reading count (ascending):\\n";
        for (const Sensor& sensor : sensors)
            sensor.printInfo();

    } catch (const exception& e) {
        cerr << "Error: " << e.what() << '\\n';
        return 1;
    }
    return 0;
}
```